

STUDENT MANAGEMENT SYSTEM

Academic Project

A Console-Based Java Application

Project Presentation

Java OOP • ArrayList • Constructors • Encapsulation

1. Project Overview

Console-based Java application for managing student records.

Users can Add Student, View Students, or Exit.

Each student is represented as an object.

Student objects are stored dynamically using ArrayList.

Uses classes, constructors, encapsulation, loops, switch-case and Scanner.

2. Objectives

Store and display student information.

Demonstrate classes, objects and constructors.

Use ArrayList for dynamic storage.

Create a simple menu-driven console interface.

Separate student data from program control logic.

Provide a foundation for future development.

3. System Requirements

Hardware: Intel i3 or equivalent processor.

RAM: 2 GB minimum; 4 GB recommended.

Disk space: 50 MB free space.

OS: Windows / macOS / Linux.

Java JDK 8 or above.

IDE/Editor: VS Code, IntelliJ IDEA, Eclipse or text editor.

4. System Architecture

User Interface → displays menu and accepts input.

Control Logic → processes choices and controls the loop.

Data Model → Student class represents each student.

Collection → ArrayList stores Student objects.

Main class controls execution and uses Scanner.

5. Student Data Model

id : int — unique student identifier.

name : String — student's name.

age : int — student's age.

Student(...) constructor initializes the object.

display() prints student details.

6. Program Workflow

START

↓ Display Student Management System menu

↓ Read user's choice

→ Add Student: enter ID, name and age → create object → add to ArrayList

→ View Students: check records → display stored students

→ Exit: close Scanner and terminate

Repeat until Exit is selected.

7. Implementation Highlights

this keyword initializes Student record fields.

ArrayList allows the number of records to grow dynamically.

display() prints student information.

Main creates objects and iterates through the list.

Core concepts: Class, Object, Constructor, Encapsulation, ArrayList, Scanner, Switch-case and Loop.

8. Functional Modules

1. Add Student — accepts ID, name and age; creates and stores a Student object.
2. View Students — checks the list and displays every stored student.
3. Exit — closes Scanner and terminates the application.

9. Sample Execution

===== Student Management System =====

1. Add Student 2. View Students 3. Exit

Enter choice: 1

Enter ID: 101 Enter Name: John Doe Enter Age: 20

Student Added Successfully!

Enter choice: 2

ID: 101, Name: John Doe, Age: 20

10. Advantages & Limitations

Advantages:

- Simple and learner-friendly.
- Lightweight console application.
- Clear separation of Student and Main.
- Flexible in-memory storage using ArrayList.

Limitations:

- Data is lost when the program exits.
- Update, Delete, Search and Sorting are not implemented.
- Limited validation and console-only interface.

11. Future Scope

File / Database Storage → preserve records between sessions.

CRUD Operations → add Update and Delete capabilities.

Validation & Exception Handling → safer input.

Search & Sorting → find and organize records efficiently.

JavaFX / Swing GUI → graphical interface.

Spring Boot Web App → web-based access.

12. Learning Outcomes

Understand how Java classes model real-world entities.

Create and use objects through constructors.

Manage collections with `ArrayList`.

Build menu-driven console applications.

Apply control flow and user input practically.

Understand how a prototype can evolve into a larger software system.

13. Conclusion

The project demonstrates Java fundamentals through a practical record-management system.

It combines OOP design, constructors, collections, console input and control flow.

The session-based structure can be extended with databases, CRUD, validation, search, sorting and GUI/web interfaces.

14. References

Oracle Java Documentation

Java ArrayList Class Reference — `java.util.ArrayList`

Java Scanner Class Reference — `java.util.Scanner`